

Laravel vs React.js: Evaluating Core Web Vitals Across Server-Side and Client-Side Rendering using Google Lighthouse

Abstract—

Keywords—

I. INTRODUCTION (HEADING 1)

Aspek performa pada halaman web kini menempati posisi yang semakin strategis dalam proses pengembangan aplikasi web modern, baik dari perspektif pengguna maupun pengembang. Optimasi performa web merupakan aspek krusial dalam ekosistem digital modern yang secara langsung memengaruhi kepuasan pengguna, tingkat keterlibatan, serta keberhasilan bisnis secara keseluruhan [1]. Halaman web yang memerlukan waktu muat panjang, lamban merespons interaksi, maupun menampilkan ketidakstabilan elemen visual dapat menurunkan kenyamanan pengguna, yang pada akhirnya tercermin pada tingginya bounce rate dan melemahnya tingkat konversi. Dengan demikian, performa web telah berkembang dari sekadar urusan teknis menjadi elemen strategis yang tidak terpisahkan dari pengembangan produk digital.

Perkembangan ini semakin diperkuat melalui kebijakan Google pada tahun 2020 yang secara resmi memasukkan Core Web Vitals (CWV) ke dalam algoritma pemeringkatan SEO-nya. Seiring dengan meningkatnya kompleksitas aplikasi web, Google memperkenalkan Web Vitals sebagai seperangkat metrik terstandarisasi yang dirancang untuk mengukur kualitas pengalaman pengguna dari sisi performa pemuatan, responsivitas interaksi, maupun stabilitas visual halaman [1]. Metrik utama CWV meliputi Largest Contentful Paint (LCP), First Input Delay (FID), dan Cumulative Layout Shift (CLS). Metrik komplementer seperti Time to First Byte (TTFB), First Contentful Paint (FCP), dan Total Blocking Time (TBT) turut memberikan gambaran performa yang lebih menyeluruh.

Dalam membangun web yang cepat, salah satu keputusan paling krusial adalah memilih metode rendering. Dua metode yang paling umum dipakai saat ini adalah Server-Side Rendering (SSR) dan Client-Side Rendering (CSR). Pada metode SSR (misalnya menggunakan framework Laravel), proses pembentukan kode HTML diselesaikan di server. Alhasil, browser pengguna tinggal menampilkan halaman yang sudah jadi tanpa harus menjalankan JavaScript yang berat terlebih dahulu. Sebaliknya, metode CSR (seperti pada React.js) menyerahkan proses rendering ke browser pengguna. Browser harus mengunduh dan menjalankan kumpulan file JavaScript terlebih dahulu sebelum antarmuka halaman bisa dibangun dan ditampilkan. Lokhande et al. [2] telah membandingkan SSR dan CSR secara konseptual beserta framework-framework yang umum digunakan, namun penelitian tersebut belum menyertakan pengukuran empiris berbasis metrik performa yang terstandarisasi. Penelitian ini hadir untuk mengisi celah tersebut dengan melakukan pengujian langsung pada dua implementasi identik, satu berbasis Laravel (SSR) dan satu berbasis React.js (CSR), dengan menggunakan Google Lighthouse sebagai alat ukur Core Web Vitals secara objektif dan terukur.

Wehner et al. [3] menemukan bahwa Core Web Vitals, khususnya LCP, berkorelasi dengan kualitas pengalaman pengguna, meskipun loading time tetap menjadi faktor dominan. Namun, studi tersebut belum mengkaji secara langsung bagaimana perbedaan arsitektur rendering berkontribusi terhadap perbedaan skor CWV yang dihasilkan. Kondisi ini mendorong banyak pengembang untuk memilih metode rendering berdasarkan kebiasaan atau preferensi pribadi, bukan berdasarkan bukti kuantitatif yang terukur.

Berdasarkan celah (gap) penelitian tersebut, penelitian ini dilakukan untuk membandingkan secara langsung performa Core Web Vitals antara implementasi SSR pada Laravel dan CSR pada React.js. Pengujian ini menggunakan aplikasi web dengan konten yang sama persis dan diukur menggunakan instrumen pengukuran standar, yaitu Google Lighthouse. Penelitian ini diharapkan bisa memberikan hasil yang objektif dan berbasis data, sehingga dapat menjadi panduan praktis bagi pengembang dalam memilih arsitektur rendering yang paling sesuai untuk kebutuhan proyek mereka.

II. TINJAUAN PUSTAKA

A. Performa Web

Performa web merupakan aspek penting dalam pengembangan *website* karena berpengaruh langsung terhadap pengalaman pengguna, kepuasan, dan kualitas layanan digital [1]. *Website* dengan waktu muat yang lambat dapat menurunkan kenyamanan pengguna serta mengurangi interaksi pengguna terhadap sistem [2]. Selain itu, optimasi performa *website* secara berkelanjutan dapat meningkatkan kualitas pengalaman pengguna dan efektivitas akses informasi pada aplikasi berbasis web modern.

Menurut Jain [1], performa web yang baik berkontribusi langsung terhadap peningkatan kualitas layanan digital dan pengalaman pengguna dalam mengakses aplikasi berbasis web. Selain itu, van Riet, Malavolta, dan Ghaleb [2] menyatakan bahwa optimasi performa web secara berkelanjutan penting dilakukan untuk meningkatkan efisiensi sistem serta menjaga kualitas interaksi pengguna terhadap *website*.

B. Core Web Vitals (CWV)

Core Web Vitals (CWV) merupakan standar pengukuran performa *website* yang digunakan untuk mengevaluasi pengalaman pengguna berdasarkan kecepatan pemuatan halaman, respons interaksi, dan kestabilan tampilan visual [1]. Pengukuran ini membantu pengembang dalam mengidentifikasi masalah performa yang dapat memengaruhi kenyamanan pengguna saat mengakses *website*. Tiga indikator utama dalam *Core Web Vitals* terdiri dari *Largest Contentful Paint* (LCP), *First Input Delay* (FID), dan *Cumulative Layout Shift* (CLS) yang digunakan untuk mengukur performa pemuatan konten, responsivitas, dan stabilitas visual halaman web [1].

Wehner dkk. [3] menyatakan bahwa *Largest Contentful Paint* (LCP) memiliki korelasi positif terhadap kualitas pengalaman pengguna (*Quality of Experience/QoE*), meskipun waktu *loading* secara keseluruhan tetap menjadi faktor yang paling dominan dalam menentukan kualitas pengalaman pengguna saat mengakses *website*. Selain itu, Wehner dkk. [4] juga menjelaskan bahwa metrik *Core Web Vitals* dapat digunakan sebagai indikator evaluasi performa *website modern*, namun hasil pengukurannya tetap perlu dipertimbangkan bersama faktor pengalaman pengguna secara nyata.

C. Server-Side Rendering (SSR)

Server-Side Rendering (SSR) merupakan metode rendering halaman web yang proses pembentukan tampilannya dilakukan di sisi server sebelum dikirim ke browser dalam bentuk dokumen HTML lengkap. Pendekatan ini memungkinkan halaman dapat ditampilkan lebih cepat karena browser tidak perlu melakukan rendering awal menggunakan JavaScript secara penuh [5].

Beberapa penelitian menunjukkan bahwa pendekatan SSR memiliki performa loading awal yang lebih baik dibandingkan CSR. Iskandar et al. [6] menyatakan bahwa situs web berbasis SSR menghasilkan waktu *First Contentful Paint* (FCP) yang lebih cepat karena proses rendering dilakukan di sisi server sebelum halaman dikirim ke browser. Gangishetti dan Jain [7] juga menunjukkan hasil yang serupa, bahwa SSR cenderung menghasilkan nilai *Largest Contentful Paint* (LCP) dan *First Contentful Paint* (FCP) yang lebih baik dibandingkan implementasi CSR yang setara, terutama pada jaringan lambat atau perangkat dengan spesifikasi rendah. Selain itu, pendekatan SSR juga mempermudah proses *indexing* oleh mesin pencari karena konten halaman telah tersedia sejak awal halaman dimuat [6].

D. Client-Side Rendering (CSR)

Client-Side Rendering (CSR) merupakan pendekatan rendering halaman web yang proses pembentukan tampilannya dilakukan di sisi klien menggunakan JavaScript. Pada pendekatan ini, browser akan memproses dan membangun tampilan halaman setelah menerima file JavaScript dari server [8].

Beberapa penelitian menunjukkan bahwa CSR memiliki keunggulan dalam interaktivitas dan responsivitas aplikasi karena perubahan tampilan dapat dilakukan tanpa memuat ulang seluruh halaman. Iskandar et al. [6] menyatakan bahwa pendekatan CSR lebih sesuai digunakan pada aplikasi web dinamis yang membutuhkan interaksi pengguna secara real-time. Selain itu, Gangishetti dan Jain [7] menjelaskan bahwa CSR memungkinkan proses navigasi halaman berlangsung lebih fleksibel setelah proses rendering awal selesai dilakukan.

Namun demikian, pendekatan CSR memiliki kelemahan pada performa pemuatan awal halaman karena browser harus memproses file JavaScript terlebih dahulu sebelum konten dapat ditampilkan secara penuh. Kondisi tersebut menyebabkan nilai *First Contentful Paint* (FCP) dan *Largest Contentful Paint* (LCP) pada CSR cenderung lebih lambat dibandingkan SSR, terutama pada perangkat dengan spesifikasi rendah atau jaringan lambat [6][7].

E. Framework Laravel

Laravel adalah *framework* PHP populer untuk pengembangan aplikasi web modern yang menerapkan arsitektur MVC (*Model-View-Controller*) [1]. Secara default, Laravel menggunakan Blade sebagai *template engine* yang mendukung *server-side rendering* (SSR), memungkinkan penyisipan data *backend* langsung ke struktur HTML sebelum dikirim ke browser [9].

Fahrudin et al. (2025) menunjukkan bahwa Blade unggul dalam kecepatan *loading* awal dan kompatibilitas SEO, namun mengalami keterbatasan signifikan dalam skalabilitas—pada skenario data besar (10.000 entri), Total Blocking Time (TBT) melonjak drastis hingga 2.790 ms, melampaui ambang toleransi pengguna 10 detik yang direkomendasikan Jakob Nielsen [9]. Temuan ini mengindikasikan bahwa meskipun Blade efisien untuk aplikasi berbasis konten statis, pendekatan ini kurang memadai untuk sistem yang membutuhkan interaktivitas tinggi dengan volume data besar.

F. Library React.js

React.js adalah *library* JavaScript yang dikembangkan oleh Meta untuk membangun antarmuka pengguna interaktif dan dinamis melalui pendekatan *client-side rendering* (CSR) [6]. Dalam CSR, browser menerima HTML *skeleton* minimal beserta file JavaScript, kemudian *framework* seperti React membangun tampilan secara dinamis di sisi klien menggunakan virtual DOM [5].

Keunggulan utama React terletak pada stabilitas interaktivitasnya pada skala data besar. Penelitian Fahrudin et al. (2025) menunjukkan bahwa React hanya mencatat TBT sebesar 70 ms pada 10.000 entri, jauh lebih rendah dibandingkan Blade yang mencapai 2.790 ms pada volume data yang sama [9]. Meskipun React memiliki kelemahan berupa waktu *loading* awal yang lebih lambat, keunggulan responsivitas *post-load* menjadikannya pilihan yang lebih tepat untuk aplikasi berbasis data dengan kebutuhan interaktivitas tinggi [9].

G. Google Lighthouse

Google Lighthouse adalah alat audit otomatis untuk mengevaluasi kualitas halaman web berdasarkan metrik *Core Web Vitals* yang terdiri dari *Largest Contentful Paint* (LCP), *First Input Delay* (FID), dan *Cumulative Layout Shift* (CLS) [7].

Menurut McGill et al. (2023), Lighthouse digunakan sebagai *tools open-source* untuk pengukuran performa dan audit aksesibilitas guna meningkatkan kualitas *website*, namun memiliki keterbatasan dalam menangani *input* berganda dan *format output* laporan yang terbatas [8]. Penelitian Wehner et al. (2022) juga menemukan bahwa *Core Web Vitals* memiliki nilai prediktif yang lebih rendah terhadap *Quality of Experience* (QoE) pengguna dibandingkan waktu pemuatan halaman dan *Speed Index* tradisional [7].

III. METODOLOGI

Before you begin to format your paper, first write and save the content as a separate text file. Complete all content and organizational editing before formatting. Please note sections A-D below for more information on proofreading, spelling and grammar.

Keep your text and graphic files separate until after the text has been formatted and styled. Do not use hard tabs, and limit use of hard returns to only one return at the end of a paragraph. Do not add any kind of pagination anywhere in the paper. Do not number text heads-the template will do that for you.

A. Abbreviations and Acronyms

Define abbreviations

B. Units

- Use either SI (MKS) or CGS as primary units. (SI units are encouraged.)

Use a long dash rather than a hyphen for a minus sign. Punctuate equations with commas or periods when they are part of a sentence, as in:

$$a + b = \gamma \tag{1}$$

Note that the equation is centered using a center tab stop.

IV. HASIL DAN PEMBAHASAN

After the text edit has been completed, the paper is ready for the template.

A. Authors and Affiliations

The template is designed for, but not limited to, six authors.

1) For papers with less than six authors: To change the default, adjust the template as follows.

a) Selection: Highlight all author and affiliation lines.

B. Identify the Headings

Headings, or heads, are organizational devices that guide the reader through your paper. There are two types: component heads and text heads.

a) Insert figures and tables after they are cited in the text. Use the abbreviation “Fig. 1”, even at the beginning of a sentence.

TABLE I. TABLE TYPE STYLES

Table Head	Table Column Head		
	Table column subhead	Subhead	Subhead
copy	More table copy ^a		

^a Sample of a Table footnote. (Table footnote)

Fig. 1. Example of a figure caption. (figure caption)

Figure Labels: Use 8 point

ACKNOWLEDGMENT (Heading 5)

The preferred spelling of the word “acknowledgment” in America is without an “e” after the “g”. Avoid the stilted expression “one of us (R. B. G.) thanks ...”. Instead, try “R. B. G. thanks...”. Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

The template will number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first ...”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors’ names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

jjmi

M mnj

[1] 1] V. Jain, “International Journal of Core Engineering & Management ISSN No : 2348-9510,” *Int. J. Core Eng. Manag.*, vol. 7, no. 06, pp. 198–205, 2023.

[2] [2] J. Lokhande, S. Mantri, and Y. Hande, “A Comparative Analysis of Client Side Rendering and Server Side Rendering,” 2023.

[3] [3] N. Wehner, M. Amir, M. Seufert, R. Schatz, and T. Hobfeld, “A Vital Improvement? Relating Google’s Core Web Vitals to Actual Web QoE,” in *2022 14th International Conference on Quality of Multimedia Experience, QoMEX 2022, Institute of Electrical and Electronics Engineers Inc.*, 2022. doi: 10.1109/QoMEX55416.2022.9900881.

[4] [4] J. van Riet, I. Malavolta, and T. A. Ghaleb, “Optimize along the way: An industrial case study on web performance,” *Journal of Systems and Software*, vol. 198, p. 111593, Apr. 2023, doi: 10.1016/J.JSS.2022.111593.

[5] [5] M. Fahrudin, T. F. Kusumasari, and E. N. Alam, “A Performance Comparison of Frontend Rendering Strategies in Laravel Applications,” *2025 9th International Conference On Electrical, Electronics And Information Engineering (ICEEIE)*, pp. 1–6, 2025, doi: 10.1109/iceeie66203.2025.11252452.

[1] V. Jain, “WEB VITALS AND CORE METRICS FOR WEB PERFORMANCE OPTIMIZATION,” *Int. J. Core Eng. Manag.*, vol. 7, no. 06, pp. 198–205, 2023.

[2] J. Lokhande, S. Mantri, and Y. Hande, “A Comparative Analysis of Client Side Rendering and Server Side Rendering,” 2023.

[3] N. Wehner, M. Amir, M. Seufert, R. Schatz, and T. Hoßfeld, “A Vital Improvement ? Relating Google ’ s Core Web Vitals to Actual Web QoE,” 2022, doi: 10.1109/QoMEX55416.2022.9900881.

[4] J. van Riet, I. Malavolta, and T. A. Ghaleb, “Optimize along the way: An industrial case study on web performance,” *J. Syst. Softw.*, vol. 198, p. 111593, 2023, doi: 10.1016/j.jss.2022.111593.

[5] “Server-Side Rendering (SSR) - Inertia.js Documentation.” Accessed: May 07, 2026. [Online]. Available: <https://inertiajs.com/docs/v3/advanced/server-side->

rendering

- [6] T. Fadhilah Iskandar, M. Lubis, T. Fabrianti Kusumasari, and A. Ridho Lubis, "Comparison between client-side and server-side rendering in the web development," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 801, no. 1, 2020, doi: 10.1088/1757-899X/801/1/012136.
- [7] S. Gangishetti and V. Jain, "Comparative Analysis of Client-Side vs. Server-Side Rendering for Large-Scale Content Platforms," *Int. J. AI, BigData, Comput. Manag. Stud.*, vol. 7, no. 1, pp. 74–80, 2026, doi: 10.63282/3050-9416.IJAIBDCMS-V7I1P112.
- [8] "Rendering: Client-side Rendering (CSR) | Next.js." Accessed: May 07, 2026. [Online]. Available: <https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering>
- [9] M. Fahrudin, T. F. Kusumasari, and E. N. Alam, "A Performance Comparison of Frontend Rendering Strategies in Laravel Applications," *2025 9th Int. Conf. Electr. Electron. Inf. Eng.*, pp. 1–6, 2025, doi: 10.1109/iceeie66203.2025.11252452.

